Тема 7. Потоки (Stream). Метод pipe()

хекслет колледж

Цель занятия:

Научиться обрабатывать большие объемы данных (файлы, сетевые запросы), не перегружая оперативную память сервера.

хекслет колледж

Учебные вопросы:

1. Концепция потоков: Зачем они нужны? Типы потоков.
2. Readable Stream: Как забрать данные
3. Writable Stream: Как отправить данные
4. Метод .pipe() - «Соединяем трубы»
5. Практическое применение: Стриминг видео/аудио

хекслет колледж

1. Концепция потоков. Типы потоков.

В чем проблема обычного подхода?
Когда мы используем fs.readFile(), Node.js пытается считать файл целиком в оперативную память (RAM).
Если файл весит 2 ГБ, а у вашего сервера свободно всего 1 ГБ, программа завершится с ошибкой JavaScript heap out of memory.
Даже если памяти хватает, сервер будет «заблокирован» на несколько секунд, пока читает файл, и не сможет отвечать другим пользователям.

хекслет колледж

Что такое поток (Stream)?

Поток - это абстрактный интерфейс для работы с данными по частям (chunks).

Вместо того чтобы ждать загрузки всего «бассейна» данных, мы открываем «кран» и обрабатываем каждую упавшую каплю (порцию данных) немедленно.

Преимущества:

1. Эффективность по памяти: Вы можете обработать файл любого размера (даже 100 ГБ), используя всего 10–20 МБ оперативной памяти.
2. Эффективность по времени: Обработка (например, сжатие или отправка по сети) начинается сразу, как только пришел первый «чанк», не дожидаясь конца чтения.

кекслет колледж

Типы потоков в Node.js

Все потоки в Node.js являются экземплярами класса EventEmitter, поэтому они работают на событиях. Существует 4 основных типа:

хекслет колледж

1. Readable (Потоки чтения)

Источник, из которого мы забираем данные.

- Примеры: fs.createReadStream() (чтение файла), reprocess.stdin q (входящий запрос в HTTP-сервере), (ввод в терминале).
- Ключевое событие: 'data' (пришла новая порция данных).

хекслет колледж

2. Writable (Потоки записи)

Место назначения, куда мы отправляем данные.

- Примеры: fs.createWriteStream() (запись в файл), res (ответ HTTP-сервера), process.stdout (вывод в консоль).
- Ключевой метод: .write().

хекслет колледж

3. Duplex (Двусторонние)

Поток, который является одновременно и Readable, и Writable. Вход и выход независимы.

- Пример: net.Socket (TCP-соединение). Вы можете одновременно отправлять данные и получать их.

хекслет колледж

4. Transform (Преобразующие)

Разновидность Duplex-потоков, которые изменяют данные «на лету» в процессе передачи.

- Примеры: zlib.createGzip() (сжатие данных), crypto.createCipheriv() (шифрование).
- Суть: Данные входят в одну сторону, меняются и выходят из другой.

хекслет колледж

Сравнительная таблица: Buffer vs Stream

Характеристика	Буфер (readFile)	Поток (createReadStream)
Память	Занимает объем, равный размеру файла	Занимает фиксированный малый объем (64 КБ)
Скорость старта	Ждет полной загрузки	Начинает работу мгновенно
Ограничения	Ограничен объемом RAM	Практически не ограничен

хекслет колледж

Итог:
«Потоки - это не способ хранения данных, это способ их транспортировки.
Если данных много или они приходят постепенно (как видео из интернета), мы используем Streams».

Главная идея: «Не жди всё, работай с тем, что уже пришло».

хекслет колледж

2. Readable Stream: Как забрать данные

Что такое Chunk (Чанк)?
Когда мы читаем файл через поток, Node.js не выдает его по одному символу.
Он выдает данные порциями, которые называются chunks (куски).

- По умолчанию размер одного чанка в Node.js - 64 КБ.
- Чанк обычно приходит в виде объекта Buffer (сырые бинарные данные).

Если мы хотим работать с ним как с текстом, нужно указать кодировку.

хекслет колледж

Два режима работы потока

Потоки могут находиться в одном из двух состояний:

1. **Paused** (Приостановлен): Данные не текут сами по себе. Мы должны явно просить их (например, методом `.read()`).
2. **Flowing** (Текущий): Данные передаются максимально быстро, как только они становятся доступны. Этот режим включается автоматически, как только мы добавляем обработчик события `'data'`.

хекслет колледж

Основные события Readable Stream

Работа с потоком чтения строится на прослушивании четырех ключевых событий:

Событие	Когда срабатывает?	Что передает в callback?
'data'	Когда пришел новый чанк данных.	Сам чанк (chunk)
'end'	Когда данные закончились (файл дочитан).	—
'error'	Если произошла ошибка (файла нет, нет прав).	Объект ошибки (err)
'open'	Когда файл успешно открыт (только для файлов).	Дескриптор файла (fd)

кекслет колледж

Пример:

```
const fs = require("fs");
const path = require("path");

const stream = fs.createReadStream(path.join(__dirname, 'war_and_peace.txt'), {
    encoding: 'utf8',       // Чтобы чанки были строками, а не Buffer
    highWaterMark: 16 * 1024 // Изменяем размер чанка на 16 КБ (для теста)
});

stream.on("data", function (chunk) {
  console.log("--- Новый чанк ---");
  console.log(chunk.length);
  console.log(chunk);
});
```

кекслет колледж

Метод fs.createReadStream()

Этот метод создает и возвращает объект Readable Stream, который связан с конкретным файлом.

Метод принимает два аргумента:

- path (строка или Buffer): Путь к файлу. Рекомендуется всегда использовать path.join(), чтобы избежать проблем с путями в разных ОС.
- options (объект): Набор настроек (необязательно).

хекслет колледж

Основные свойства объекта настроек:

- encoding: По умолчанию null (возвращает данные в виде Buffer). Если указать 'utf8', Node.js будет автоматически декодировать байты в текст.
- highWaterMark: Размер «порции» (чанка) в байтах. По умолчанию - 64 КБ.
- start / end: Позволяют читать не весь файл, а только определенный диапазон байт (удобно для докачки файлов). start: Номер байта, с которого нужно начать (отсчет с 0). end: Номер байта, на котором нужно закончить (включительно!).

кекслет колледж

Что возвращает?

Он возвращает объект класса fs.ReadStream.

В отличие от fs.readFile, этот метод не читает файл сразу. Он лишь «подготавливает трубу».

Чтение начнется только тогда, когда вы:

1. Добавите обработчик события 'data'.
2. Вызовете метод .pipe().
3. Или вызовете метод .resume().

хекслет колледж

Что за объект fs.ReadStream?

Когда вы пишете:

```
const stream = fs.createReadStream(...)
```

вы создаете «умную трубу». Этот объект не содержит в себе весь файл сразу.

Он умеет постепенно доставать данные из файла и передавать их вам.

хекслет колледж

У этого объекта есть встроенные методы для управления процессом:

- stream.pause() - «Пауза». Остановить чтение данных. Полезно, если программа не успевает их обрабатывать.
- stream.resume() - «Пуск». Продолжить чтение после паузы.
- stream.destroy() - «Уничтожить». Немедленно закрыть поток и освободить файл.
- stream.pipe(destination) - «Перенаправить». Автоматически передать данные из этого потока в другой (например, сразу в файл записи).

кекслет колледж

Что за метод .on()?

Поскольку Node.js - событийная среда, мы не ждем, пока функция что-то вернет. Мы подписываемся на события. Метод .on() - это «ухо» вашего скрипта.

- Первый аргумент - название события (строка, например, "data").
- Второй аргумент - функция (callback), которая сработает, когда это событие произойдет.

Аналогия: Это как заказать пиццу и сказать: «Когда (on) курьер приедет (событие), позвони (функция) мне». Вы не стоите у двери, вы занимаетесь своими делами, пока не сработает событие.

хекслет колледж

Разбор конструкции stream.on()

```
stream.on("data", function (chunk) {
    // Этот код сработает многократно
});
```

"data" - это стандартное имя события. Оно означает: «Внутренний буфер заполнился данными (на 64 КБ), и я готов их отдать».
function (chunk) - это функция, которую Node.js вызовет автоматически.
chunk (чанк) - это аргумент, в который Node.js положит кусочек данных, считанный в этот конкретный момент.

кекслет колледж

Почему chunk?

Если ваш файл весит 1 МБ, а размер чанка по умолчанию - 64 КБ, то событие "data" сработает примерно 16 раз.
Внутри функции вы будете получать по 64 КБ данных за один раз.

Как это работает пошагово:

- Node.js читает первые 64 КБ → срабатывает on("data") → в chunk попадает первая часть → вы её обрабатываете.
- Node.js читает следующие 64 КБ → снова срабатывает on("data") → в chunk попадает вторая часть...
- ...и так до конца файла.

хекслет колледж

Краткая шпаргалка по событиям

Помимо "data", у объекта stream есть еще три важных «сигнала»:

- stream.on("end", ...) - сработает один раз, когда файл закончится.
- stream.on("error", ...) - сработает, если файл не найден или произошел сбой.
- stream.on("open", ...) - сработает в самом начале, когда файл только-только открылся.

хекслет колледж

Пример (полный контроль):

```
const fs = require('fs');
const path = require('path');

// // Создаем поток чтения
const stream = fs.createReadStream(path.join(__dirname, 'war_and_peace.txt'), {
    encoding: 'utf8',      // Чтобы чанки были строками, а не Buffer
    highWaterMark: 16 * 1024 // Изменяем размер чанка на 16 КБ (для теста)
});

// 1. Подписываемся на получение данных
stream.on('data', (chunk) => {
    console.log('--- Новый чанк ---');
    console.log(chunk.length); // размер чанка в символах
    console.log(chunk);        // фрагмент файла (чанк)
});
```

кекслет колледж

```
// 2. Обработка завершения
stream.on('end', () => {
    console.log('Чтение файла завершено успешно.');
});

// 3. Обработка ошибок
stream.on('error', (err) => {
    console.error('Произошла ошибка:', err.message);
});
```

хекслет колледж

Итог:

- Поток - это не данные, это способ их доставки.
- Объект stream - это Readable Stream.
- У него есть методы для контроля (pause, resume, destroy).
- Метод .on() используется для подписки на события.
- Событие "data" - это способ получать файл по частям (чанками), не забивая память сервера.
- Чанки - это «порции» данных, которые позволяют работать с гигантскими файлами по чуть-чуть.

хекслет колледж

3. Writable Stream: Как отправить данные

Что такое Writable Stream?

Это поток, предназначенный для записи данных в определенное место (файл, ответ сервера, терминал).
В Node.js он представлен объектом класса fs.WriteStream, который мы создаем методом fs.createWriteStream().
В отличие от потока чтения, который «кричит» нам через события, когда данные готовы, в поток записи мы сами отправляем данные, когда нам это удобно.

хекслет колледж

Пример:

```
const fs = require('fs');
const path = require('path');

let writeableStream = fs.createWriteStream('writeTest.txt');

writeableStream.write('Hello\n');
writeableStream.write('World\n');
writeableStream.write('!!!!!\n');

writeableStream.end();
```

кекслет колледж

Основные методы управления:

1. stream.write(chunk): Главный метод. Отправляет порцию данных в поток.
   - Можно вызывать многократно.
   - Можно передавать строку или Buffer.
2. stream.end([chunk]): Сообщает потоку, что данных больше не будет.
   - После вызова этого метода записывать что-то в поток уже нельзя.
   - Если передать чанк в качестве аргумента, он будет записан последним перед закрытием.
3. stream.destroy(): Немедленно закрывает поток, отменяя все незавершенные операции записи.

хекслет колледж

Основные события Writable Stream

Событие	Когда срабатывает?
'finish'	Срабатывает только после вызова .end(), когда все данные физически записаны на диск.
'error'	Если произошла ошибка (например, нет прав на запись или диск переполнен).
'drain'	Сигнализирует о том, что внутренний буфер потока пуст и он готов принимать новые данные (важно для контроля скорости).

кекслет колледж

Пример:

```
const fs = require('fs');
const path = require('path');

// Создаем "трубу" для записи в файл
const writeStream = fs.createWriteStream(path.join(__dirname, 'output.txt'), {
    encoding: 'utf8'
});

// Записываем данные по частям
writeStream.write('Строка 1\n');
writeStream.write('Строка 2\n');
writeStream.write('Строка 3\n');
writeStream.write('Строка 4\n');
writeStream.write('Строка 5\n');
```

кекслет колледж

Пример:

```
// Завершаем запись
writeStream.end('Конец файла.');

// Узнаем, когда файл реально сохранился на диск
writeStream.on('finish', () => {
    console.log('Файл успешно записан и закрыт.');
});

// Обработка ошибок
writeStream.on('error', (err) => {
    console.error('Ошибка записи:', err.message);
});
```

кекслет колледж

Важно запомнить:

- У Readable (Чтение) - событие 'end' (источник иссяк).
- У Writable (Запись) - событие 'finish' (работа по записи завершена).

хекслет колледж

4. Метод .pipe() - «Соединяем трубы»

В чем главная проблема ручного копирования?

Если мы будем вручную читать данные через .on('data') и сразу записывать их через .write(), мы столкнемся с проблемой разности скоростей.

Пример: Мы читаем файл с быстрого SSD, а записываем на медленную флешку или отправляем в интернет по слабому каналу.

Результат: Данные будут накапливаться в оперативной памяти быстрее, чем успевать уходить. Это приведет к переполнению RAM и «падению» процесса.

хекслет колледж

Что такое .pipe()?

Метод .pipe() (в переводе «труба») - это встроенный механизм Node.js, который связывает поток чтения (Readable) с потоком записи (Writable).

Синтаксис:

```
readStream.pipe(writeStream);
```

хекслет колледж

3 причины использовать .pipe():

- **Автоматизация**: Не нужно вручную слушать события 'data', 'end' и 'error'.

- **Управление памятью** (Backpressure): Метод сам следит за скоростью. Если поток записи занят, .pipe() приостанавливает поток чтения и возобновляет его только тогда, когда место освободится.

- **Автозавершени**е: Как только в источнике закончатся данные, .pipe() сам вызовет метод .end() у приемника, корректно закрыв файл.

кекслет колледж

Пример: Копирование файла

```
const fs = require('fs');
const path = require('path');

const src = fs.createReadStream(path.join(__dirname, 'video.mp4'));
const dest = fs.createWriteStream(path.join(__dirname, 'video_copy.mp4'));

// Магия: данные текут автоматически и безопасно для памяти
src.pipe(dest);

// Слушаем событие завершения на приемнике
dest.on('finish', () => {
    console.log('Файл успешно скопирован!');
});
```

хекслет колледж

Цепочки потоков (Chaining)

Метод .pipe() возвращает поток назначения.

Благодаря этому можно строить целые конвейеры обработки данных.

Хекслет колледж

Пример, Чтение -> Сжатие -> Запись в файл:

```
const fs = require('fs');
const path = require('path');
const zlib = require('zlib'); // Встроенный модуль для архивации

const readStream = fs.createReadStream(path.join(__dirname, 'war_and_peace.txt'));
const writeStream = fs.createWriteStream(path.join(__dirname, 'war_and_peace.txt.gz'));
const gzip = zlib.createGzip(); // Поток трансформации

// Соединяем три "трубы"
readStream
    .pipe(gzip)         // Пропускаем через "фильтр" сжатия
    .pipe(writeStream); // Записываем результат

// Слушаем событие завершения на приемнике
writeStream.on('finish', () => {
    console.log('Файл успешно архивирован!');
});
```

кекслет колледж

Итог:

- .pipe() - это «умный мост» между чтением и записью.
- Главное преимущество - автоматический контроль потребления памяти (Backpressure).
- Если вам нужно просто передать данные из точки А в точку Б без сложной логики в процессе - всегда используйте .pipe().

хекслет колледж

Выводы:

- Потоки vs Буферы: Вместо того чтобы загружать файл целиком в RAM (fs.readFile), мы обрабатываем его по частям (chunks).
- Потоки позволяют держать потребление памяти стабильно низким (обычно около 64 КБ на поток)
- Работа с Readable Stream строится на подписке на события. Главное из них - 'data', которое приносит новую порцию данных.
- Метод .pipe(): Это «золотой стандарт» соединения потоков. Он автоматически управляет скоростью передачи (Backpressure)
- Типы потоков:
  - Readable - только чтение.
  - Writable - только запись.
  - Transform - изменение данных «на лету» (например, сжатие).

кекслет колледж

Контрольные вопросы

- Что произойдет с Node.js процессом, если мы попытаемся считать файл размером 8 ГБ через fs.readFile() на сервере с 4 ГБ оперативной памяти?
- Почему размер чанка в событии 'data' не всегда равен значению highWaterMark?
- В чем принципиальная разница между событиями 'end' и 'finish'? К каким типам потоков они относятся?
- Почему при копировании файлов из одного места в другое метод src.pipe(dest) лучше, чем ручная подписка на src.on('data')?
- Как использовать fs.createReadStream, чтобы прочитать только последние 100 байт файла? (Подсказка: параметры start и end).

хекслет колледж